

Student Name/ID Number/Section	
HTU Course Number and Title	30201400 Systems Programming
BTEC Unit Code and Title	
Academic Year	2023-2024 Summer
Assignment Author	Samer Suleiman
Course Tutor	Samer Suleiman
Assignment Title	Text File analyses and Manipulation
Assignment Ref No	1
Issue Date	01/08/2024
Formative Assessment dates	From 02/08/2024 to 22/08/2024
Submission Date	03/09/2024
IV Name & Date	Abdullah Al-Amaren 31/07/2024

STUDENT ASSESSMENT SUBMISSION AND DECLARATION

When submitting evidence for assessment, each student must sign a declaration confirming that the work is their own

Student name:	Assessor name: Samer Suleiman	
Issue date: 1/08/2024	Submission date: 29/08/2024	Submitted on:

Programme: Computing

HTU Course Name: System Programming

BTEC Course Title:

HTU Course Code : 30201400

BTEC Course Code :

Assignment number and title: 1, Text File analyses and Manipulation

Plagiarism:

Plagiarism is a particular form of cheating. Plagiarism must be avoided at all costs and students who break the rules, however innocently, may be penalized. It is your responsibility to ensure that you understand **correct referencing practices**. As a university level student, you are expected to use appropriate references throughout and keep carefully detailed notes of all your sources of materials for material you have used in your work, including any material downloaded from the Internet. Please consult the relevant unit lecturer or your course tutor if you need any further advice.

I certify that the assignment submission is entirely my own work and I fully understand the consequences of plagiarism. I understand that making a false declaration is a form of malpractice.

Student:

Student Signature:

Date:

As an employee at XYZ Company, you were tasked with troubleshooting problems related to the text file analysis and manipulation functions of the system. Before delving into the basic data analysis task, you were first required to solve problems related to other system functions, such as addition, multiplication, and subtraction. These basic tasks were critical to ensuring the overall reliability and accuracy of the system before more complex data analysis techniques could be applied.

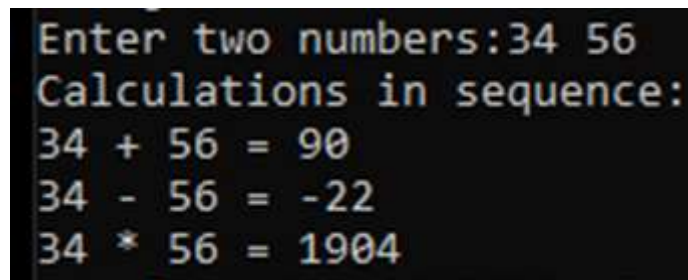
[illegible]

P1 What the System Programming and show its relationships with the OS?
P2 Define multiprogramming and describe why and how can we applied.
P3 Describe -with aid of a diagram- the communication abilities of different processes.
P4 Discuss the usage of shell scripting in comparison with C language in terms of system services.
P5 Define multitasking and describe why and how can we apply.
P6 Implement a client-server system component to handle UDP/TCP communications.

The code below represents a demonstration of a calculator project that runs in sequence, based on it answer the following questions:

```
1  #include <stdio.h>
2
3  //=====
4  void sum(int n1, int n2)
5  {
6      int s=n1 + n2;
7      printf("%d + %d = %d\n",n1,n2,s);
8  }
9  //=====
10 void sub(int n1, int n2)
11 {
12     int s=n1 - n2;
13     printf("%d - %d = %d\n",n1,n2,s);
14 }
15 //=====
16 void mul(int n1, int n2)
17 {
18     int s=n1 * n2;
19     printf("%d * %d = %d\n",n1,n2,s);
20 }
21 //=====
22 int main()
23 {
24     int x,y;
25     printf("Enter two numbers:");
26     scanf("%d%d",&x,&y);
27     sum(x,y);
28     sub(x,y);
29     mul(x,y);
30 }
31
```

- **Output:**



```
Enter two numbers:34 56
Calculations in sequence:
34 + 56 = 90
34 - 56 = -22
34 * 56 = 1904
```

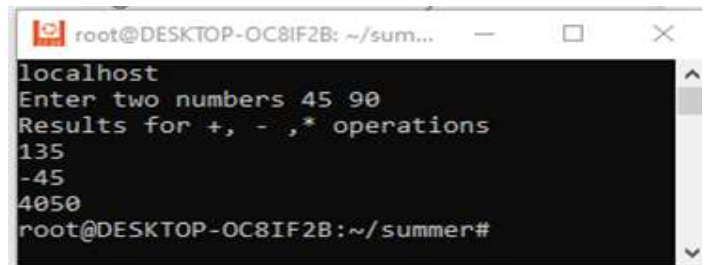
1. Discuss the multi-process concepts by explaining the necessary changes on the program to make it execute in parallel and produce similar output above. (You can rewrite the program).
2. Discuss the multithreading concepts by explaining the necessary changes on the program to make it execute concurrently and produce similar output above. (You can rewrite the program).

3. Rewrite the above calculator program as a client-server application in which the server reads two integers from the client and returns the results of the summation, subtraction, and multiplication, the clients get the results and prints them. You will be given the skeleton code for the client-server.

A terminal window titled 'root@DESKTOP-OC8IF2B: ~/su...' showing the server's execution. The output is: 'Calculator server is waiting....', 'ClientIP address is: 127.0.0.1', 'Calculator server reads Client', 'Calculator server terminated', and the prompt 'root@DESKTOP-OC8IF2B:~/summer#'.

```
root@DESKTOP-OC8IF2B: ~/su...
Calculator server is waiting....
ClientIP address is: 127.0.0.1
Calculator server reads Client
Calculator server terminated
root@DESKTOP-OC8IF2B:~/summer#
```

Terminal 1: Server

A terminal window titled 'root@DESKTOP-OC8IF2B: ~/sum...' showing the client's execution. The output is: 'localhost', 'Enter two numbers 45 90', 'Results for +, -, * operations', '135', '-45', '4050', and the prompt 'root@DESKTOP-OC8IF2B:~/summer#'.

```
root@DESKTOP-OC8IF2B: ~/sum...
localhost
Enter two numbers 45 90
Results for +, -, * operations
135
-45
4050
root@DESKTOP-OC8IF2B:~/summer#
```

Terminal 2: Client

4. Explain the function of the **accept ()** function in server-side socket programming. What does it return, and what are its main parameters?

5. The above code represents a demonstration for a calculator project that is written in C language and distributed in 3 different source files: “sum.c”, “sub.c” and “mul.c” as appear in the figures below.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  //=====
4  void sub(int n1, int n2)
5  {
6      int s=n1 - n2;
7      printf("%d - %d = %d\n",n1,n2,s);
8  }
9  //=====
10 int main(int arg, char *args[])
11 {
12     int x,y;
13     x=atoi(args[1]);
14     y=atoi(args[2]);
15     sub(x,y);
16 }
```

sum.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  //=====
4  void sum(int n1, int n2)
5  {
6      int s=n1 + n2;
7      printf("%d + %d = %d\n",n1,n2,s);
8  }
9  //=====
10 int main(int arg, char *args[])
11 {
12     int x,y;
13     x=atoi(args[1]);
14     y=atoi(args[2]);
15     sum(x,y);
16 }
```

sub.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  //=====
4  void mul(int n1, int n2)
5  {
6      int s=n1 * n2;
7      printf("%d * %d = %d\n",n1,n2,s);
8  }
9  //=====
10 int main(int arg, char *args[])
11 {
12     int x,y;
13     x=atoi(args[1]);
14     y=atoi(args[2]);
15     mul(x,y);
16 }
```

mul.c

Based on the previous three programs, answer the following questions (You can rewrite the program as a shell script):

- a) Discuss how to use the Shell script to run the same programs or calculations.

b) How differ the shell script from the C language to execute the three operations.

c) If one of the three programs has syntax what will happen in the case of shell script and the C language.

6. Consider the following snippet of code using fork:

```
1 #include<stdio.h>
2 #include<unistd.h>
3 int main(){
4 pid_t pid;
5 pid=fork();
6 if(pid==0){
7 int c=5;
8 c+=5;
9 }else if (pid>0){
10 pid=fork();
11 c+=10;
12 if(pid){
13 c+=10;
14 }
15 }
16
17 printf("%d \n",c);
18
19 }
```


a) Draw a picture of the hierarchical process tree that is created by running this program.

b) Assume that you have modified the previous code and added `fork()` on line 16. Redraw the hierarchical process tree.

7. Based on the following program:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5
6 int main() {
7
8     char *original_message = (char *)malloc(20 * sizeof(char));
9
10
11     strcpy(original_message, "Hello, World!");
12
13     printf(" original_message:%s\n", original_message);
14
15     int original_fd = dup(STDOUT_FILENO);
16
17
18     write(original_fd, original_message, strlen(original_message));
19
20     free(original_message);
21
22     return 0;
23 }
24 }
```

- a) Explain the concept of system programming. Additionally, identify the system calls used in the program, describe their purposes, and explain how they relate to system programming.

- b) List other commonly used system calls in system programming and briefly explain their functions.